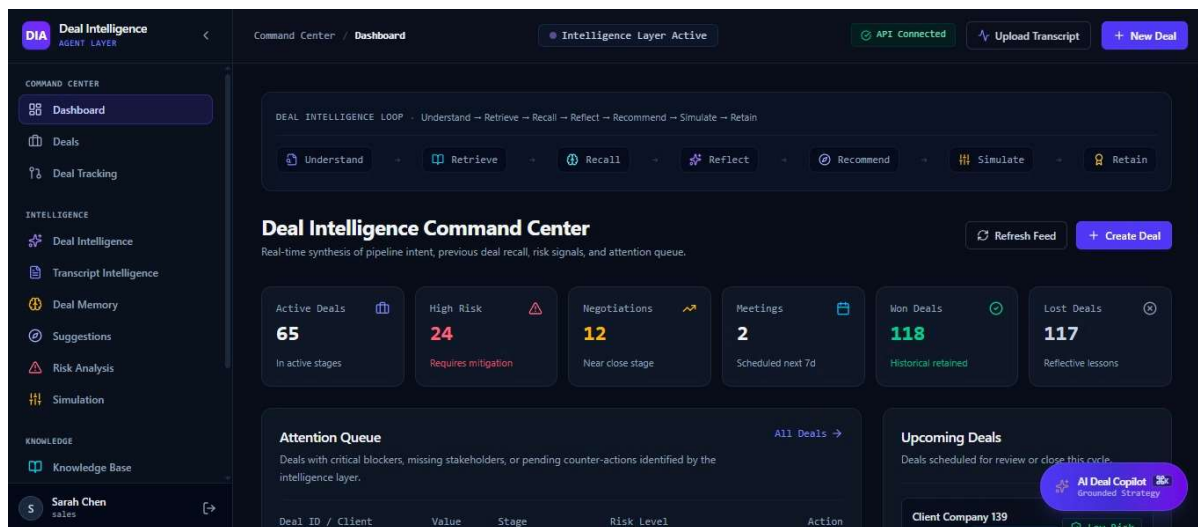


Why Vector Search Wasn't Enough for Deal Intelligence

Ask a document-search assistant "we've hit this pricing objection before, what did we do?" and it will hand you a paragraph from the pricing guide. It answers the question it can search for, not the one you asked.

That gap is the reason I stopped treating deal intelligence as a retrieval problem and started treating it as a memory problem.



Command center with the Understand-to-Retain loop

What I built

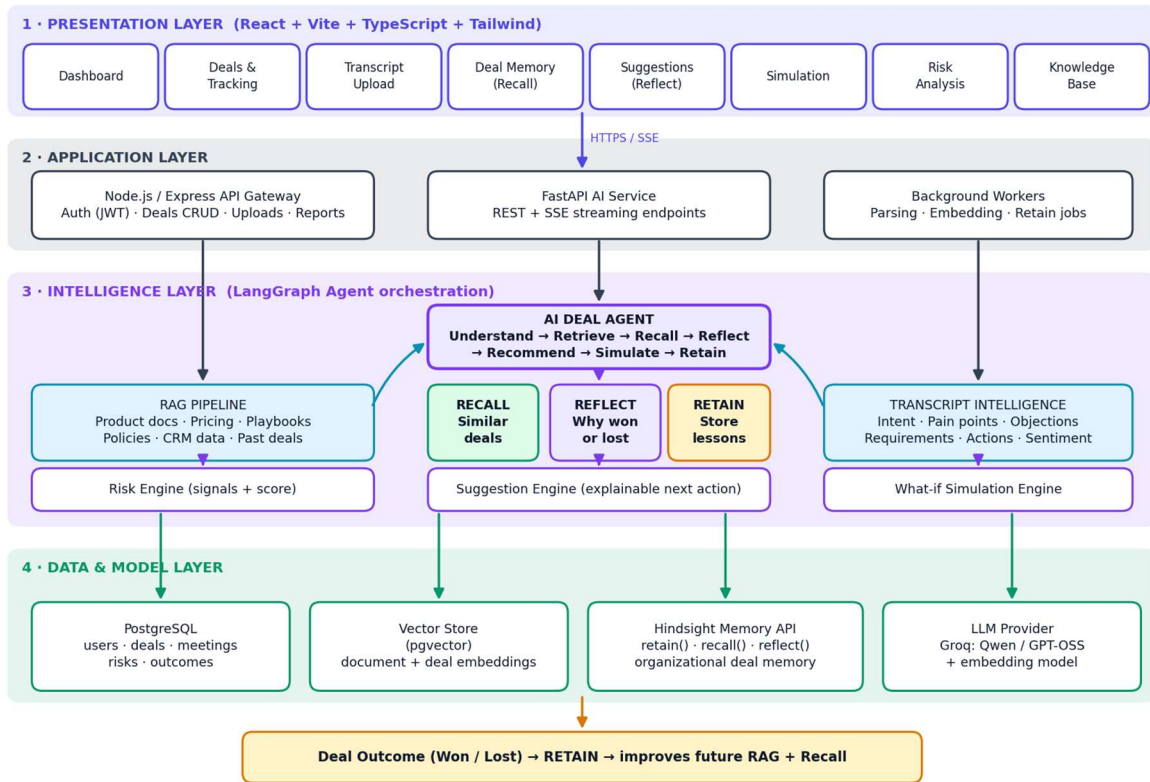
The Deal Intelligence Agent is a layer over a company's sales data. It doesn't replace the CRM. It reads what's already there (deal records, transcripts, emails, playbooks, pricing docs) and answers the questions a rep has before the next call:

- What is this customer actually worried about?
- Have we seen this before, and how did it end?
- What should I do next, and why?
- What happens if I offer a discount versus a different package?

The stack is deliberately boring: React and TypeScript, a Node.js/Express gateway, and a FastAPI service running a LangGraph agent. PostgreSQL holds structured data, pgvector holds embeddings, and a hosted LLM does the reasoning. Everything about *experience* goes through [Hindsight agent memory](#).

The agent loop has seven steps: **Understand → Retrieve → Recall → Reflect → Recommend → Simulate → Retain**. Only one of them is a search problem.

Deal Intelligence Agent: High-Level Architecture



Architecture: four layers, with Hindsight in the data layer

The repo mirrors that split: `server/`, `python-services/` and `models/artifacts`.

The screenshot shows a code editor with the following project structure on the left:

- File explorer: datasets, docs, models, python-servi..., server, db, middleware, routes, scripts, services, chatIntell..., intelligen..., pythonM..., types, src, components, hooks, lib, pages, store.

The main editor displays the `chatIntelligenceService.ts` file, which contains the following code:

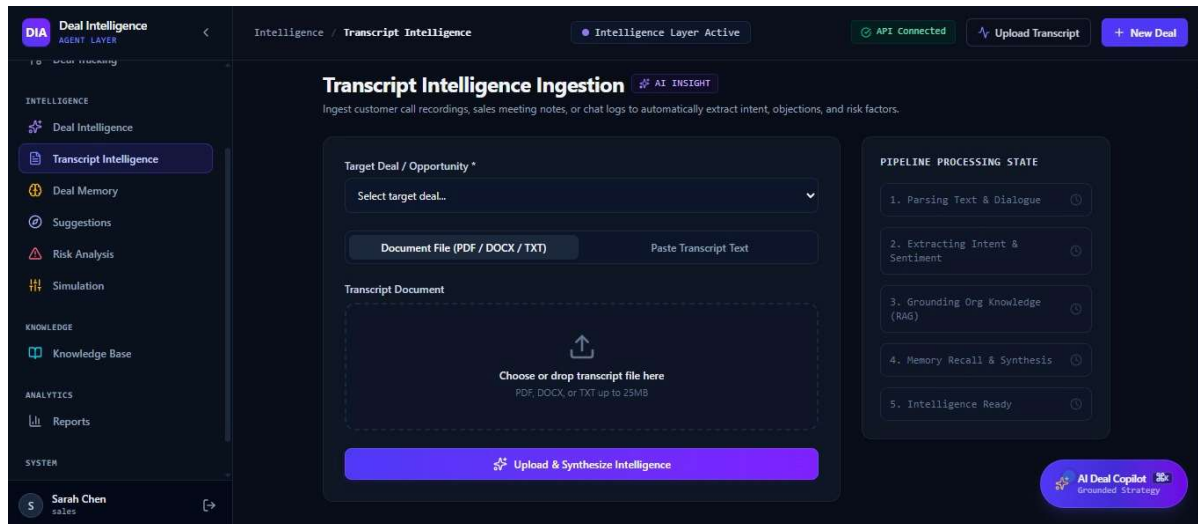
```

import { Citation } from '../types/backend/types';
4
5 export interface ChatMessageInput {
6   role: 'user' | 'assistant' | 'model';
7   content: string;
8 }
9
10 export interface ChatIntelligenceResult {
11   reply: string;
12   citations: Citation[];
13   suggestedPrompts: string[];
14   confidence: number;
15   dealContext?: {
16     id: string;
17     client: string;
18     stage: string;
19     value: number;
20     risk: string;
21     winProbability?: number;
22   };
23   modelUsed: string;
24 }
25
26 export class ChatIntelligenceService {
27   private static aiClient: GoogleGenAI | null = null;
28
29   private static getAi(): GoogleGenAI | null {
30     const key = process.env.GEMINI_API_KEY;
31     if (!this.aiClient && key && key !== 'MY_GEMINI_API_KEY' && key.trim().length > 10) {
32       this.aiClient = new GoogleGenAI({
33         apiKey: key,

```

Project structure with the chat intelligence service open

Transcripts arrive through an ingestion page (PDF, DOCX, TXT or pasted text) that extracts intent and sentiment, grounds it in organizational knowledge, then runs memory recall.



Transcript ingestion and processing pipeline

Two different questions

Retrieval-augmented generation answers one question well: *what do our documents say?* I built that first, on pgvector. It stays in the system as the knowledge base.

But a sales team's most valuable knowledge is in outcomes: the objection in the second meeting, the competitor on a call, the package that got rewritten, and the reason a deal was won or lost.

A vector index over those artifacts returns text that *resembles* your query. It can't tell you which similar deal to trust, or why the outcomes differed. Two deals can share nearly identical objection language and end in opposite ways.

So the system has two memories with different jobs:

- **Knowledge (RAG):** what our material says. The unit is a document chunk, written on upload.
- **Experience (Hindsight):** what happened last time and why. The unit is a deal outcome plus its lesson, written when the deal closes.

Retain: store the lesson, not the transcript

The obvious approach is to push every transcript into memory and let recall sort it out. That produces a large, noisy pile of text, which is RAG under a different name. I decided against it.

Retention happens at one moment instead: when a deal is closed. `POST /deals/:id/close` marks it won or lost and triggers a retain call with a distilled record: the objection, the strategy, the approach, the reason for the outcome, the competitor and the stage where it was lost.

All Hindsight access goes through a single `MemoryService`. I made that a hard rule so memory stays swappable, mockable in tests and observable.

```
# memory_service.py
class MemoryService:
    def __init__(self, client, telemetry):
        self._client = client
        self._telemetry = telemetry

    async def retain_outcome(self, company_id: str, outcome: DealOutcome) -> None:
        with self._telemetry.timed("retain", deal_id=outcome.deal_id):
            await self._client.retain(
                bank_id=f"company-{company_id}",
                content=outcome.to_lesson_text(),
                context=f"{outcome.industry} | {outcome.product} | {outcome.result}",
            )
```

Memory is scoped per company, so tenant isolation doesn't depend on a query filter. And `to_lesson_text()` writes a short narrative ("pricing objection at negotiation; restructured the package instead of discounting; won"), not a dump. Hindsight organizes and connects lessons; I make sure what goes in is worth connecting.

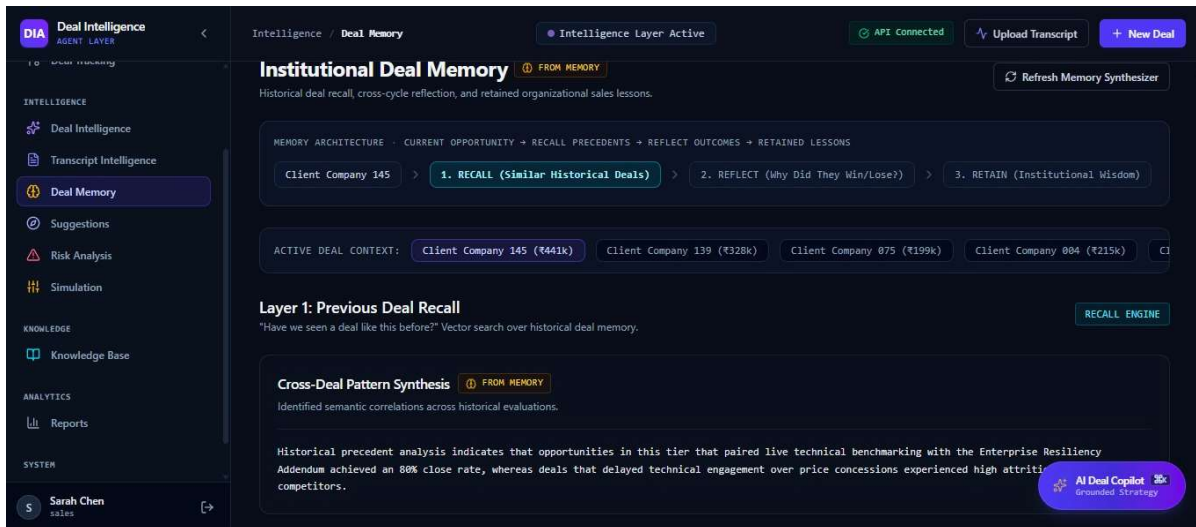
Recall: similarity is more than embeddings

A deal is similar to another when the *situation* matches: industry, deal size, product, competitors and, above all, the objections and requirements. Not when the wording overlaps.

So recall combines two signals. Structured filters from the deal record narrow the candidates, and semantic matching over objections and requirements ranks them.

```
# recall node
async def recall_node(state: DealState, memory: MemoryService) -> DealState:
    query = build_situation_query(
        objections=state.analysis.objections,
        requirements=state.analysis.requirements,
        competitor=state.deal.competitor,
        industry=state.deal.industry,
    )
    hits = await memory.recall_similar(state.deal.company_id, query, limit=3)
    state.similar_deals = [h for h in hits if h.relevance >= RELEVANCE_FLOOR]
    return state
```

`RELEVANCE_FLOOR` is the most important line in that file. Nearest-neighbor search always returns something, and nearest is not the same as near. Without a floor, the agent would reflect fluently on a deal from the wrong industry simply because it was closest.



Deal Memory: recall, reflect and retain

If nothing clears the floor, the agent stops and says so: "No sufficiently similar deal found." It then shows the evidence it does have and asks the rep for more context. I would rather ship an assistant that admits it has no experience with a situation than one that improvises an analogy.

Reflect: turn neighbors into an argument

Recall returns neighbors. It doesn't tell a rep what to do with them. Reflect compares what worked, what failed and why, and produces a short synthesis readable in seconds.

Reflect is the third `MemoryService` call:

```
# memory_service.py
async def reflect_on(self, company_id: str, deal: Deal, similar_ids: list[str]) ->
Reflection:
    with self._telemetry.timed("reflect", deal_id=deal.id):
        raw = await self._client.reflect(
            bank_id=f"company-{company_id}",
            query=build_reflect_query(deal, similar_ids),
        )
    return Reflection.model_validate_json(to_json(raw))
```

I treat reflect output as data, not prose. Every LLM output is validated against a Pydantic schema, retried once on failure, and generated at temperature 0.3 or lower.

```
# reflection schema
class Reflection(BaseModel):
    pattern: str
    what_worked: list[str]
    what_failed: list[str]
    supporting_deal_ids: list[str] = Field(min_length=1)
    confidence: Literal["low", "medium", "high"]
```

`min_length=1` on `supporting_deal_ids` has an outsized effect: a reflection that cites no deal fails validation, so the model can't hand me an unsupported opinion.

What the behavior looks like

The same question, asked of a retrieval-only assistant and the memory-backed agent, using an example manufacturing account worried about price and implementation time.

Retrieval only

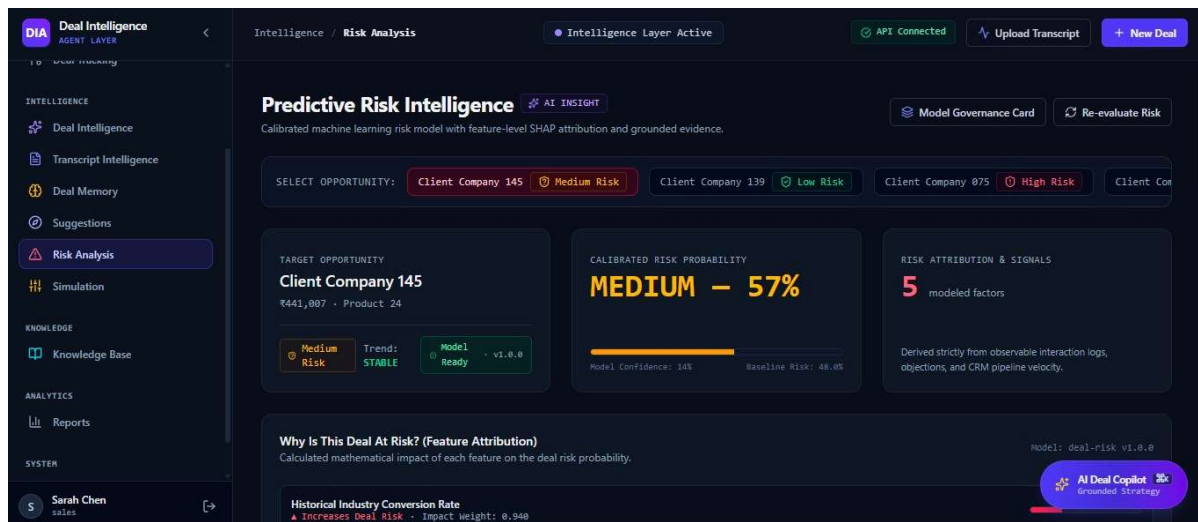
Rep: The customer is pushing back on price and implementation. What should I do?

Agent: Per the pricing guide, discounts up to 10% require manager approval. The implementation guide lists a standard onboarding timeline of...

With Recall and Reflect

Agent: Three completed deals had this same combination of concerns. One was won after the package was redesigned; the others were not. **Suggested next action:** propose a restructured package before offering a discount. *Why?* See the three cited deals. *Confidence: medium (3 similar deals).*

Risk scoring is a separate trained model over structured features such as deal value, discount ratio, win rates, unresolved objections, competitor presence and inactivity days. The UI shows each feature's contribution, so a risk level never appears without its reasons.



Risk analysis with per-feature attribution

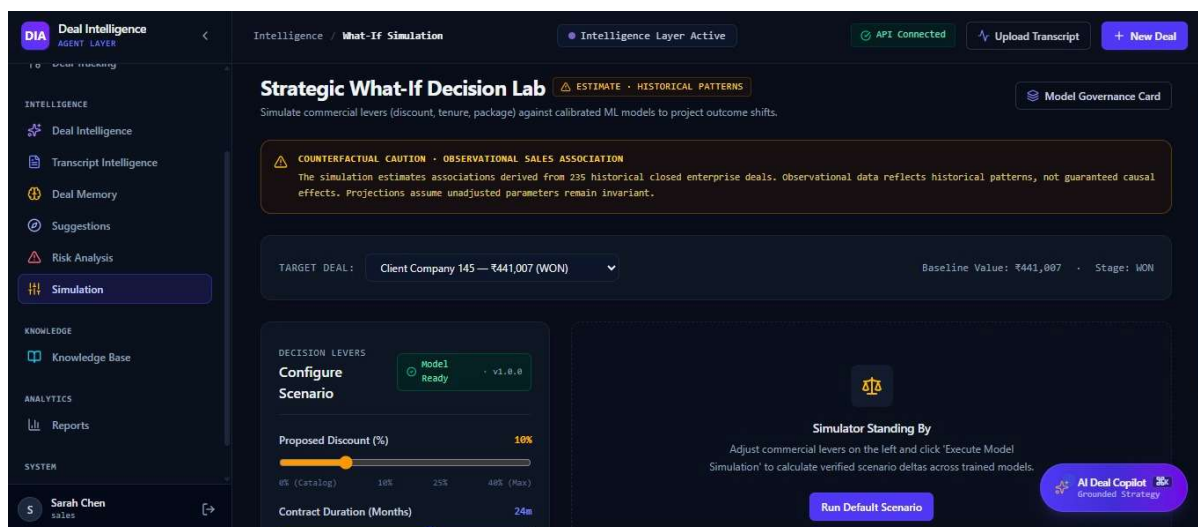
```

1 {
2   "feature_names": [
3     "log_deal_value",
4     "discount_ratio",
5     "industry_win_rate",
6     "product_win_rate",
7     "company_size_tier",
8     "total_objections",
9     "high_severity_objections",
10    "unresolved_objections",
11    "has_competitor",
12    "competitor_preferred",
13    "interaction_count",
14    "inactivity_days",
15    "rep_win_rate"
16  ],
17  "means": {
18    "log_deal_value": 12.1703,
19    "discount_ratio": 0.0504,
20    "industry_win_rate": 0.5202,
21    "product_win_rate": 0.5199,
22    "company_size_tier": 2.0,
23    "total_objections": 1.54,
24    "high_severity_objections": 0.5,
25    "unresolved_objections": 0.7,
26    "has_competitor": 0.6533,
27    "competitor_preferred": 0.18,
28    "interaction_count": 2.36,
29    "inactivity_days": 5.0,
30    "rep_win_rate": 0.5184
  }
}

```

Risk model feature list

The what-if simulator builds on the same recalled patterns. A rep adjusts three levers (discount, contract duration and package) and the agent compares scenarios against historical outcomes. Every output is labelled "est." because it's derived from past deals, not a forecast, and the agent never acts on its own. The salesperson reviews the options and decides.



What-if simulator with estimate badge

Limitations and dead ends

Retention quality sets the ceiling. A lesson distilled from a vague close reason ("customer went another way") is worthless, so closing a deal requires a structured reason, even though it adds friction.

Few deals means low confidence. Three deals are a pattern, not proof. The confidence field and "est." labels keep that visible.

Verify the API before building on it. I kept a mock `MemoryService` alongside the real one and checked every call against the [Hindsight docs](#) instead of guessing signatures. Because memory lives behind one class, corrections touch one file.

Lessons

1. **Split knowledge from experience.** Documents answer "what do we say?" Outcomes answer "what happened?" They need different storage and different failure handling.
2. **Store lessons, not logs.** [Agent memory](#) is only as good as the distilled fact you hand it. Decide what a completed unit of experience looks like.
3. **Put a floor under recall.** Nearest-neighbor search always answers, so build the "I don't know" path on purpose.
4. **Make citations a schema constraint.** If a reflection can't validate without supporting evidence, you don't have to hope the model behaves.
5. **Route memory through one door.** One service gives you mockability, tenant scoping in one place, and a log of every memory call.

A CRM records the deal. Hindsight lets the next deal start from how the last ones ended. To try the memory layer yourself, start with the [Hindsight GitHub repository](#).

Built with Hindsight. Tagging Code.in.